SPRINT 4

NIVEL 1

Descarga los archivos CSV, estúdialos y diseña una base de datos con un esquema de estrella que contenga, al menos 4 tablas de las cuales puedas realizar las siguientes consultas:

CREACIÓN DE LA BASE DE DATOS ventas_toys

1. Se crea la BBDD ventas_toys con el siguiente código:

```
# Creación Base de DAtos venta_toys

CREATE DATABASE IF NOT EXISTS ventas_toys;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	20:42:49	CREATE DATABASE IF NOT EXISTS ventas_toys	1 row(s) affected	0.016 sec

Se usa la sentencia CREATE DATABASE y se añade la cláusula condicional IF NOT EXISTS para evitar errores de “ya existe”.

2. A continuación, se crean las tablas (sólo la estructura) american_users y european_users

```
CREATE TABLE IF NOT EXISTS american_users (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    surname VARCHAR(100),
    phone VARCHAR(50),
    email VARCHAR(100),
    birth_date DATE,
    country VARCHAR(100),
    city VARCHAR(100),
    postal_code VARCHAR(20),
    address TEXT
);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	20:50:09	USE ventas_toys	0 row(s) affected	0.000 sec
2	20:50:20	CREATE TABLE IF NOT EXISTS american_users (id INT PRI...	0 row(s) affected	0.047 sec

```sql
CREATE TABLE IF NOT EXISTS european_users (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    surname VARCHAR(100),
    phone VARCHAR(50),
    email VARCHAR(100),
    birth_date DATE,
    country VARCHAR(100),
    city VARCHAR(100),
    postal_code VARCHAR(20),
    address TEXT
);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	20:50:09	USE ventas_toys	0 row(s) affected	0.000 sec
2	20:50:20	CREATE TABLE IF NOT EXISTS american_users (id INT PRI...	0 row(s) affected	0.047 sec
3	20:51:12	CREATE TABLE IF NOT EXISTS european_users (id INT PRI...	0 row(s) affected	0.031 sec

3. Comprobación 'local_infile' está ON, para poder usar otra ruta de la carpeta "Uploads" diferente a la predeterminada por MySQL.

```sql
# Comprobación 'local_infile' está ON
SHOW VARIABLES LIKE 'local_infile';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Variable_name	Value
local_infile	ON

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	21:00:57	SHOW VARIABLES LIKE 'local_infile'	1 row(s) returned	0.000 sec / 0.000 sec

4. Carga de datos de los archivos CSV a las tablas american_users, european_users:

<u>american_users</u>

```sql
LOAD DATA LOCAL INFILE 'C:/Users/ivan_/Documents/MySQL/Uploads/american_users.csv'  -- se añade LOCAL porque
INTO TABLE american_users
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'      -- se usa OPTIONALLY porque sólo algunas columnas están entre comillas
LINES TERMINATED BY '\n'        -- marca de salto de línea
IGNORE 1 ROWS                   -- la primera fila se ignora porque son los nombre de las columnas
(id, name, surname, phone, email, @v_birth_date, country, city, postal_code, address)
SET birth_date = STR_TO_DATE(@v_birth_date, '%b %e, %Y');                -- @v_birth_date es una variable
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	04:11:01	LOAD DATA LOCAL INFILE 'C:/Users/ivan_/Documents/MyS...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 ...	0.032 sec

Se usa la sentencia LOAD DATA INFILE, aunque en este caso se usa la opción LOAD DATA LOCAL INFILE ya que se carga el archivo desde el PC del analista al servidor.

Se indica a qué tabla se cargarán los datos con la palabra clave INTO TABLE.

Durante la comprobación de los datos del archivo CSV se detecta que las columnas están separadas por comas ",". Por este motivo se indica en el código de carga con cláusula FIELDS TERMINATED BY.

También se aprecia que algunas columnas están entre comillas “”, debido a esto se usa la cláusula OPTIONALLY ENCLOSED BY, de lo contrario, si todas las columnas estuvieran entre comillas se debería usar simplemente ENCLOSED BY.

Se marca el salto de línea con LINES TERMINATED BY ‘\n’.

Con la sentencia IGNORE 1 ROWS (también se puede usar LINES) se omite la primera fila ya que se tratan los nombres de los atributos.

Por último, se hace un tratamiento del formato de la fecha del archivo CSV para que coincida con el formato en MySQL. En el archivo la fecha tiene el siguiente formato: "Nov 17, 1985".

Para transformar el formato primero se llaman todos los atributos de la tabla dentro de un paréntesis, pero se sustituye el atributo ‘birth_date’ por una variable temporal indicándolo con una @ quedando así ‘@v_birth_date:

(id, name, surname, phone, email, @v_birth_date, country, city, postal_code, address)

Con la sentencia SET birth_date se indica que se va a asignar un valor a la columna llamada birth_date.

STR_TO_DATE es la función que realiza la conversión de texto a fecha.

@v_birth_date es la variable temporal donde está guardado el dato original de la fecha.

‘%b %e, %Y’ es el formato que le explica a MySQL cómo leer el texto. Donde %b es el nombre del mes abreviado (Jan, Feb, Mar...), %e día del mes en número y %Y año con cuatro dígitos.

Este proceso de conversión introduce la fecha con el formato estándar de base de datos YYYY-MM-DD.

<u>european_users</u>

```
53 •  LOAD DATA LOCAL INFILE 'C:/Users/ivan_/Documents/MySQL/Uploads/european_users.csv'
54    INTO TABLE european_users
55    FIELDS TERMINATED BY ','
56    OPTIONALLY ENCLOSED BY '"'
57    LINES TERMINATED BY '\n'
58    IGNORE 1 ROWS
59    (id, name, surname, phone, email, @v_birth_date, country, city, postal_code, address)
60    SET birth_date = STR_TO_DATE(@v_birth_date, '%b %e, %Y');
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	04:11:01	LOAD DATA LOCAL INFILE 'C:/Users/ivan_/Documents/MySQL...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 War...	0.032 sec
2	05:25:34	LOAD DATA LOCAL INFILE 'C:/Users/ivan_/Documents/MySQL...	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 War...	0.156 sec

Para la tabla european_users usamos el mismo código porque no hay diferencias en los tipos de atributos.

5. Creación de tabla intermedia ‘users’ para unificar las tablas american_users y european_users.

Este paso es necesario debido a que desde la tabla transactions no se puede tener una FOREIGN KEY vinculadas a dos tablas diferentes a la vez. Lo mismo pasa con la tabla credit_cards.

En esta nueva tabla los atributos serán los mismos, pero se añade el nuevo atributo region con el tipo de dato ENUM y se restringe los valores a EU y US.

```
CREATE TABLE IF NOT EXISTS users (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    surname VARCHAR(100),
    phone VARCHAR(50),
    email VARCHAR(100),
    birth_date DATE,
    country VARCHAR(100),
    city VARCHAR(100),
    postal_code VARCHAR(20),
    address TEXT,
    region ENUM('US', 'EU') NOT NULL    -- para definir una columna que sólo puede guardar un valor específi
);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	21:30:11	CREATE TABLE IF NOT EXISTS users (id INT PRIMARY KE...	0 row(s) affected	0.031 sec

6. Concatenar las tablas american_users y european_users con el operador UNION ALL.

En este paso se realiza la concatenación de ambas tablas e insertándolas directamente en la tabla users:

```
INSERT IGNORE INTO users
SELECT id, name, surname, phone, email, birth_date, country, city, postal_code, address, 'US'
FROM american_users
UNION ALL
SELECT id, name, surname, phone, email, birth_date, country, city, postal_code, address, 'EU'
FROM european_users;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	21:30:11	CREATE TABLE IF NOT EXISTS users (id INT PRIMARY KE...	0 row(s) affected	0.031 sec
2	21:33:18	INSERT IGNORE INTO users SELECT id, name, surname, ph...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.250 sec

Comprobamos que las filas afectadas sean correctas que en este caso es 5000.

Se puede apreciar que se usa la cláusula IGNORE dentro de la sentencia INSERT, esto se realiza para convertir errores en advertencias en el caso de ejecutar todo el script. Así mismo, se evitaría que haya un error de carga duplicada en un futuro.

7. Creación y carga de datos del resto de tablas.

credit_cards

```sql
CREATE TABLE IF NOT EXISTS credit_cards (
    id VARCHAR(50) PRIMARY KEY,         -- se usa VARCHAR porque contiene letras
    user_id INT NOT NULL,               -- se define explícitamente NOT NULL porque será una FK
    iban VARCHAR(50),
    pan VARCHAR(50),
    pin VARCHAR(4),
    cvv VARCHAR(3),
    track1 VARCHAR(100),
    track2 VARCHAR(100),
    expiring_date DATE,

    CONSTRAINT fk_cards_users
        FOREIGN KEY (user_id) REFERENCES users(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:15:22	CREATE TABLE IF NOT EXISTS credit_cards (id VARCHAR(...	0 row(s) affected	0.046 sec

Aspectos claves en la creación de esta tabla son:

Se usa el tipo VARCHAR para la id que es PK porque los valores contienen números y letras.

Dejar indicado cuáles son las PK y FK, en el caso de la FK, user_id, se define explícitamente que no puedo ser un valor nulo, por eso, NOT NULL.

Se define la FOREING KEY y se referencia a la tabla y el atributo vinculados con la regla CONSTRAINT.

Se usan las restricciones de FOREING KEYS, ON DELETE CASCADE y ON UPDATE CASCADE para mantener la integridad referencial entre la tabla padre users y la tabla hija credit_cards.

```sql
LOAD DATA LOCAL INFILE 'C:/Users/ivan_/Documents/MySQL/Uploads/credit_cards.csv'
INTO TABLE credit_cards
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(id, user_id, iban, pan, pin, cvv, track1, track2, @v_expiring_date)
SET expiring_date = STR_TO_DATE(@v_expiring_date, '%m/%d/%y');
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:15:22	CREATE TABLE IF NOT EXISTS credit_cards (id VARCHAR(...	0 row(s) affected	0.046 sec
2	22:39:56	LOAD DATA LOCAL INFILE 'C:/Users/ivan_/Documents/MyS...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 ...	0.266 sec

Para cargar los datos del archivo CSV en la tabla credit_cards se cambia el formato del atributo expiring_date. Por este motivo se crea la variable temporal @v_expiring_date para después usar la sentencia SET junto con el atributo expiring_date y la función

STR_TO_DATE para transformar los datos correspondientes a la fecha de caducidad en el archivo CSV con el siguiente formato MM/DD/YYYY, por eso se indica %m/%d/%Y.

<u>companies</u>

```sql
CREATE TABLE IF NOT EXISTS companies (
    company_id VARCHAR(20) PRIMARY KEY,
    company_name VARCHAR(150),
    phone VARCHAR(30),
    email VARCHAR(150),
    country VARCHAR(100),
    website VARCHAR(255)
);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:15:22	CREATE TABLE IF NOT EXISTS credit_cards (id VARCHAR(50) ...	0 row(s) affected	0.046 sec
2	22:39:56	LOAD DATA LOCAL INFILE 'C:/Users/ivan_/Documents/MySQL...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 War...	0.266 sec
3	23:58:00	CREATE TABLE IF NOT EXISTS companies (company_id VARC...	0 row(s) affected	0.063 sec

```sql
LOAD DATA LOCAL INFILE 'C:/Users/ivan_/Documents/MySQL/Uploads/companies.csv'
INTO TABLE companies
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(company_id, company_name, phone, email, country, website);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:15:22	CREATE TABLE IF NOT EXISTS credit_cards (id VARCHAR(50) ...	0 row(s) affected	0.046 sec
2	22:39:56	LOAD DATA LOCAL INFILE 'C:/Users/ivan_/Documents/MySQL...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 War...	0.266 sec
3	23:58:00	CREATE TABLE IF NOT EXISTS companies (company_id VARC...	0 row(s) affected	0.063 sec
4	23:59:36	LOAD DATA LOCAL INFILE 'C:/Users/ivan_/Documents/MySQL...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnin...	0.047 sec

<u>products</u>

```sql
CREATE TABLE IF NOT EXISTS products (
    id INT PRIMARY KEY,
    product_name VARCHAR(100),
    price DECIMAL(10,2),
    colour CHAR(7),
    weight DECIMAL(4,2),
    warehouse_id VARCHAR(10)
);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	00:01:28	CREATE TABLE IF NOT EXISTS products (id INT PRIMARY ...	0 row(s) affected	0.046 sec

```sql
LOAD DATA LOCAL INFILE 'C:/Users/ivan_/Documents/MySQL/Uploads/products.csv'
INTO TABLE products
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(id, product_name, @price_raw, colour, weight, warehouse_id)
SET price = CAST(SUBSTRING(@price_raw, 2) AS DECIMAL(10,2));
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	00:01:28	CREATE TABLE IF NOT EXISTS products (id INT PRIMARY ...	0 row(s) affected	0.046 sec
2	00:02:06	LOAD DATA LOCAL INFILE 'C:/Users/ivan_/Documents/MyS...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 War...	0.031 sec

En esta ocasión para poder cargar los datos del archivo CSV con la máxima compatibilidad, se realiza una limpieza y transformación del dato en el atributo precio: ej. $161.11.

Para ello se crea una variable temporal @price_raw que contiene el dato original.

A continuación, se usa la función SUBSTRING(@price_raw, 2) para cortar el texto empezando desde la segunda posición con el objetivo de eliminar el primer carácter $.

Después, se utiliza la función CAST(... AS DECIMAL(10,2)) que toma el texto resultante y lo transforma en un número decimal de longitud 10 y 2 decimales.

Por último, con la sentencia SET se fija el resultado limpio y convertido en la columna price.

8. Creación de la tabla transactions. Modelo final y creación de relaciones.

```sql
CREATE TABLE IF NOT EXISTS transactions (
    id VARCHAR(50) PRIMARY KEY,
    card_id VARCHAR(50) NOT NULL,
    company_id VARCHAR(50) NOT NULL,    -- aquí cambiamos el nombre del atributo buisness_id del csv por com
    timestamp DATETIME,
    amount DECIMAL(10,2) NOT NULL,      -- NOT NULL porque una transacción sin importe no tiene sentido
    declined BOOLEAN,
    product_ids TEXT,
    user_id INT NOT NULL,
    lat DOUBLE,
    longitude DOUBLE,
```

Esta es la primera parte del código para crear la tabla transactions, en este punto, cabe resaltar que en la estructura de la tabla se deja indicado explícitamente que los atributos card_id, company_id y user_id deben ser valor NOT NULL porque serán las FOREIGN KEYS por las que se vincularán a otras tablas padre. También se indica que amount es del tipo NOT NULL porque una transacción sin importe no tiene sentido.

Además, para que el esquema sea más coherente y homogeneizar criterios se modifica el atributo business_id por company_id.

```sql
    CONSTRAINT fk_transactions_card
        FOREIGN KEY (card_id)
        REFERENCES credit_cards(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,

    CONSTRAINT fk_transactions_user
        FOREIGN KEY (user_id)
        REFERENCES users(id)
        ON DELETE RESTRICT              -- Se usa RESTRICT porque no se quiere borrar el usuario y conservar
        ON UPDATE CASCADE,

    CONSTRAINT fk_transactions_company
        FOREIGN KEY (company_id)
        REFERENCES companies(company_id)
        ON DELETE RESTRICT              -- Igual que con users no se quiere perder el histórico de compras c
        ON UPDATE CASCADE
);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	01:56:06	CREATE TABLE IF NOT EXISTS transactions (id VARCHAR(...	0 row(s) affected	0.094 sec

En esta segunda parte del código se definen las FOREING KEY y se referencia a las tablas y los atributos vinculados con la regla CONSTRAINT.

Se usan las restricciones de FOREING KEYS, ON DELETE CASCADE y ON UPDATE CASCADE para mantener la integridad referencial entre las tablas padre e hijas. En este caso se usa también ON DELETE RESTRICT porque se quiere conservar el histórico de transacciones de usuarios y el histórico de compras de empresas.

```sql
LOAD DATA LOCAL INFILE 'C:/Users/ivan_/Documents/MySQL/Uploads/transactions.csv'
INTO TABLE transactions
FIELDS TERMINATED BY ';'                -- En este csv las columnas están separadas por ; y no por comas com
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(id, card_id, company_id, @v_timestamp, amount, declined, product_ids, user_id, @v_lat, @v_longitude)
SET
    timestamp = STR_TO_DATE(@v_timestamp, '%Y-%m-%d %H:%i:%s'),
    lat = CAST(@v_lat AS DECIMAL(18,15)),
    longitude = CAST(@v_longitude AS DECIMAL(18,15));
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	01:56:06	CREATE TABLE IF NOT EXISTS transactions (id VARCHAR(...	0 row(s) affected	0.094 sec
2	02:20:25	LOAD DATA LOCAL INFILE 'C:/Users/ivan_/Documents/MyS...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped:...	6.766 sec

Para la carga de datos desde el archivo CSV detecta que las columnas están separadas por “;” y no por “,” como en las tablas anteriores.

Se crean tres variables temporales para transformar el formato de los atributos timestamp, lat y longitude.

9. Creación tabla intermedia para separar productos de una misma transacción.

```sql
CREATE TABLE IF NOT EXISTS transaction_products (
    transaction_id VARCHAR(50),
    product_id INT,
    PRIMARY KEY (transaction_id, product_id),

    CONSTRAINT fk_tp_transaction
        FOREIGN KEY (transaction_id)
        REFERENCES transactions(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,

    CONSTRAINT fk_tp_product
        FOREIGN KEY (product_id)
        REFERENCES products(id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	01:56:06	CREATE TABLE IF NOT EXISTS transactions (id VARCHAR(...	0 row(s) affected	0.094 sec
2	02:20:25	LOAD DATA LOCAL INFILE 'C:/Users/ivan_/Documents/MyS...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped:...	6.766 sec
3	02:34:42	CREATE TABLE IF NOT EXISTS transaction_products (trans...	0 row(s) affected	0.062 sec

Esta tabla es necesaria para la resolución del último ejercicio. En el archivo CSV se puede comprobar que en la columna del atributo product_ids existen más de un producto por transacción (una lista dentro de una columna), con esta estructura será imposible hacer JOINS correctamente, es decir, rompe el modelo relacional.

Por este motivo, se crea una tabla intermedia para normalizar el modelo, se separa la lista en una tabla nueva que se denominará transaction_products. Para crear la tabla hay que tener en cuenta que la PRIMARY KEY será compuesta por el atributo transaction_id y el producto_id y las FOREING KEY estarán conectadas a transactions.id y products.id.

Carga de los datos:

```sql
INSERT INTO transaction_products (transaction_id, product_id)
SELECT
    t.id,
    CAST(TRIM(SUBSTRING_INDEX(SUBSTRING_INDEX(t.product_ids, ',', n.n), ',', -1)) AS UNSIGNED)
FROM transactions t
JOIN (
    SELECT 1 n UNION SELECT 2 UNION SELECT 3 UNION SELECT 4 UNION SELECT 5
) n
ON CHAR_LENGTH(t.product_ids) - CHAR_LENGTH(REPLACE(t.product_ids, ',', '')) >= n.n -1
WHERE t.product_ids IS NOT NULL
AND t.product_ids != '';
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	01:56:06	CREATE TABLE IF NOT EXISTS transactions (id VARCHAR(...	0 row(s) affected	0.094 sec
2	02:20:25	LOAD DATA LOCAL INFILE 'C:/Users/ivan_/Documents/MyS...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped:...	6.766 sec
3	02:34:42	CREATE TABLE IF NOT EXISTS transaction_products (trans...	0 row(s) affected	0.062 sec
4	04:34:32	INSERT INTO transaction_products (transaction_id, product_i...	253391 row(s) affected Records: 253391 Duplicates: 0 Wami...	8.000 sec

Para entender este código se explicará por pasos:

a) La tabla destino INSERT INTO transaction_products (transaction_id, producto_id)
b) El SELECT que genera las filas Esta línea extrae el n elemento.
    - SUBSTRING_INDEX(t.product_ids, ',', n.n) toma los primeros n elementos.
    - SUBSTRING_INDEX(..., ',', -1) toma el último elemento
    - CAST(... AS UNSIGNED) convierte el texto en número
c) JOIN (SELECT 1 n ...) n esto crea una tabla temporal. Permite extraer hasta 5 product_ids por transacción. Si tiene menos el filtro del JOIN lo evita.
d) ON CHAR_LENGTH (...) – CHAR_LENGTH(...) >= n.n -1 esto cuenta cuántas comas tiene el campo. Sólo extrae el elemento n si existen al menos n elementos. Así evita errores como intentar extraer el 4º elemento cuando sólo hay 3. Funciona como filtro que evita extraer más elementos de los que existen.
e) Filtro final WHERE t.products_ids IS NOT NULL AND t.products_ids != '' Evita procesar filas vacías.

10. Limpieza del modelo

```
248 •  ALTER TABLE transactions DROP COLUMN product_ids;
249
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
4	04:34:32	INSERT INTO transaction_products (transaction_id, product...	253391 row(s) affected Records: 253391 Duplicates: 0 War...	8.000 sec
5	05:30:59	ALTER TABLE transactions DROP COLUMN product_ids	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.078 sec

Se elimina la columna producto_ids de la tabla transactions para que no de problemas con los JOINS.

```
250 •  DROP TABLE american_users;
251
252 •  DROP TABLE european users;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	05:34:41	DROP TABLE american_users	0 row(s) affected	0.016 sec
2	05:34:47	DROP TABLE european_users	0 row(s) affected	0.032 sec

Se eliminan las tablas american_users y european_users.

MODELO FINALIZADO Y NORMALIZADO

A continuación, se procede a realizar las consultas.

Ejercicio 1

Realiza una subconsulta que muestre todos los usuarios con más de 80 transacciones utilizando al menos 2 tablas.

```
SELECT u.id, u.name, u.surname, COUNT(t.id) AS total_transacciones
FROM users u
JOIN transactions t
ON t.user_id = u.id
GROUP BY u.id
HAVING COUNT(t.id) > 80;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	name	surname	total_transacciones
185	Molly	Gilliam	110
289	Dxwgi	Hwcru	94
318	Bnyr	Astuw	91
454	Sfzzoh	Xgvfridxs	81

Result 1 | Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	21:51:04	SELECT u.id, u.name, u.sumame, COUNT(t.id) AS total_trans...	4 row(s) returned	0.078 sec / 0.000 sec

La clave aquí es usar HAVING y no WHERE porque no puede filtrar por agregaciones, y HAVING sí.

Ejercicio 2

Muestra la media de amount por IBAN de las tarjetas de crédito en la empresa Donec Ltd, usa al menos dos tablas.

```
SELECT cc.iban, AVG(t.amount) AS media_amount
FROM transactions t
JOIN credit_cards cc ON t.card_id = cc.id
JOIN companies c ON t.company_id = c.company_id
WHERE c.company_name = 'Donec Ltd'
GROUP BY cc.iban;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

iban	media_amount
XX9114064011255863075868 05	356.246667
SK9446370242474562577506	142.960000
XX7767529178459529755556 40	257.370000
XX4138273622897193049089 90	139.590000
XX3477872460707696107803 08	240.410000

Result 2 | Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	21:51:04	SELECT u.id, u.name, u.sumame, COUNT(t.id) AS total_trans...	4 row(s) returned	0.078 sec / 0.000 sec
2	01:27:15	SELECT cc.iban, AVG(t.amount) AS media_amount FROM tra...	371 row(s) returned	0.063 sec / 0.000 sec

NIVEL 2

Ejercicio 1

Crea una nueva tabla que muestre el estado de las tarjetas de crédito basado en si las tres últimas transacciones han sido rechazadas entonces es inactiva, si al menos una no es rechazada entonces es activa. Partiendo de esta tabla responde: ¿Cuántas tarjetas están activas?

Para este ejercicio se opta por crear una VISTA (VIEW) card_status en vez de una TABLA ya que una VIEW siempre se mantiene actualizada y es más dinámica.

```sql
CREATE VIEW card_status AS                  -- aquí se crea la vista card_status
WITH ultimas AS (                           -- en este punto se utiliza una CTE temporal
    SELECT
        t.card_id,
        t.declined,
        ROW_NUMBER() OVER (                 -- se usa una window function
            PARTITION BY t.card_id
            ORDER BY t.timestamp DESC
        ) AS rn
    FROM transactions t
)
SELECT                                      -- se sigue creando la VIEW
    u.card_id,
    CASE                                    -- se crea la lógica condicional y la columna status
        WHEN SUM(u.declined) = COUNT(*) THEN 'INACTIVA' -- si al contar todas son declined entonces inactiva
        ELSE 'ACTIVA'
    END AS status                           -- columna status
FROM ultimas u                              -- tabla temporal creada la CTE
JOIN credit_cards cc                        -- se usa inner join para obtener sólo las tarjetas con transacc
    ON cc.id = u.card_id
WHERE u.rn <= 3                             -- se filtra por las 3 últimas transacciones
GROUP BY u.card_id;                         -- y se agrupan por id de tarjeta
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	03:06:31	USE ventas_toys	0 row(s) affected	0.000 sec
2	03:06:57	CREATE VIEW card_status AS-- aquí se crea la vista card_sta...	0 row(s) affected	0.000 sec

1. Con la sentencia CREATE VIEW se crea la vista.

2. WITH ultimas AS (...) es la CTE temporal (Common Table Expression) que servirá para dar un conjunto de resultados temporal que se usará en la vista card_status. Es decir, la tabla ultimas no existe físicamente, se está creando "al vuelo" dentro del WITH.

3. Se usa una window function ROW_NUMBER() OVER(...) para crear una columna 'rn' con filas según fechas (1 = más reciente). Numera filas dentro de una partición.

- Todas las transacciones de esta tarjeta (PARTITION BY card_id)
- Ordenadas por fecha (ORDER BY timestamp DESC)

Esto asigna un número 1, 2, 3...

4. Una vez definida la CTE se sigue creando la VIEW y se crea la lógica condicional con CASE y la columna status.

WHEN SUM(u.declined) = COUNT(*) THEN 'INACTIVA'

Esta línea del código lo que dice es: Si al contar todas son declined entonces está inactiva.

5. Se usa JOIN para obtener sólo las tarjetas con transacciones.

6. WHERE u.rn <= 3 es el filtro que selecciona sólo las 3 últimas transacciones por tarjeta.

Una vez creada la VIEW card_status se procede a hacer la consulta: ¿Cuántas tarjetas están activas?

```
319 •  SELECT COUNT(*) AS tarjetas_activas
320    FROM card_status
321    WHERE status = 'ACTIVA';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

tarjetas_activas
4995

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	03:12:34	CREATE VIEW card_status AS-- aquí se crea la vista card_st...	0 row(s) affected	0.000 sec
2	03:14:05	SELECT COUNT(*) AS tarjetas_activas FROM card_status W...	1 row(s) returned	0.266 sec / 0.000 sec

NIVEL 3

Crea una tabla con la que podamos unir los datos del nuevo archivo products.csv con la base de datos creada, teniendo en cuenta que desde transaction tienes product_ids. Genera la siguiente consulta:

Ejercicio 1 (tablas ya modificadas en el modelo al principio del documento)

Necesitamos conocer el número de veces que se ha vendido cada producto.

```
334 •  SELECT
335        p.id AS product_id,
336        p.product_name,
337        COUNT(tp.product_id) AS veces_vendido
338    FROM products p
339    JOIN transaction_products tp
340    ON p.id = tp.product_id
341    GROUP BY p.id, p.product_name
342    ORDER BY veces_vendido DESC;
343
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	product_name	veces_vendido
52	riverlands the duel	2654
29	Tully maester Tarly	2635
21	duel Direwolf	2609
16	the duel warden	2608
66	mustafar jinn	2601

Result 5 | Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
7	04:06:45	SELECT COUNT(tp.product_id), p.product_name FROM tra...	100 row(s) returned	0.235 sec / 0.000 sec
8	04:11:33	SELECT p.id AS product_id, p.product_name, COUN...	100 row(s) returned	0.235 sec / 0.000 sec